

后训练学习计划 · 第一周（重制版 v2）

本周定位：**Phase 0 启动周——把地基一次打牢**。主题是 GPU 环境 + micrograd 亲手复刻 + 手写 attention + 知识自测。本版按你新的节奏重排：每天投入 **≥6 小时**，一天吸收原计划约 1.5 天的内容，全周 7 天（6 天学习 + 1 天缓冲验收），总预算约 41.5 小时、花费 ≤¥30。

用法说明：每天先看「今日总览」，再从「任务 1」开始逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗红字 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	GPU 环境全通 + micrograd 视频(上)	wandb 出现第一条记录	6h
Day 2 周二	micrograd 视频(下) + 完整跟写	mlp.py 在 moons 上 acc≥95%	6.5h
Day 3 周三	盲写复刻 + 数值梯度对拍	mine.py 收敛 + GitHub 链接	6h
Day 4 周四	限时自测 6 题 + 错题映射 + 博客成文	week1-gaps.md + 博客草稿	6h
Day 5 周五	手写 attention(单头→多头→可视化)	attention.py 单测全绿 + 博客发布	6h
Day 6 周六	PyTorch 桥接 + 对拍 + 周复盘	torch 对照版代码 + review.md	7h
Day 7 周日	缓冲补漏 + 总验收 + 下周预习	验收 ≥6/7 项通过	4h

注：GPU 花费只有 Day 1-2 需要（4090 约 ¥1.6/小时 × 共 4-5 小时 ≈ ¥8），加上充值手续费，全周 ≤¥30 绰绰有余。Day 3 起的本地写码、Day 5 的 attention 都可以在本地 CPU 完成。

注：每天结束前花 10 分钟把卡点写进 `NOTES.md`（见 Day 1 任务 5），这些是 Day 4 博客的原材料，别欠账。

二、开始前：目录与工具（5 分钟）

- 在本地磁盘建一个代码目录（后续所有代码都放这里）：

```
mkdir -p E:\study\posttrain && cd E:\study\posttrain
mkdir day1 day2 day3 day4 day5 day6
```
- 本周你会用到的账号（Day 1 任务 1 注册）：AutoDL（租 GPU）、wandb（训练记录）、Hugging Face、魔搭 ModelScope。建议用同一个邮箱+密码管理器记录。

Day 1 · 周一：GPU 环境全通 + micrograd 视频（上）

今日总览（6h）：今天只有一个目标——**晚上 10 点前，你的 wandb 账号里出现一条从 GPU 实例发出的训练记录**。上午注册账号 + 租卡 + 环境冒烟（约 2h），下午开始看 Karpathy 的 micrograd 视频上半场（约

3.5h)，最后 10 分钟写收工笔记。micrograd 是后训练一切知识的起点：反向传播不是魔法，是一个链式法则记账系统，本周你要亲手复刻它。

任务 1：注册账号四件套（约 30 分钟）

1. 打开 `https://www.autodl.com` → 注册 → 实名认证（支付宝扫码即可）。
2. 打开 `https://wandb.ai` → Sign up（可直接用 Google 账号登录）→ 登录后进入 `https://wandb.ai/authorize`，**复制页面上显示的 API Key**（一串 32 位字符），先粘到记事本里。
3. 打开 `https://huggingface.co` → 注册（邮箱验证）。国内如打不开，跳过，本周不强制。
4. 打开 `https://modelscope.cn` → 注册（淘宝/支付宝账号可直接登录）。

完成判据：四个网站都能正常登录；wandb API Key 已保存到记事本。

注：Hugging Face 打不开不影响本周任务，不要在这上面耗超过 10 分钟。魔搭是它的国内平替。

任务 2：租一台 4090 并连上终端（约 40 分钟）

1. AutoDL 控制台 → 算力市场 → 地区选「西北 B 区」或任意有货的 → 卡型筛选 **RTX 4090 (24G)** → 计费选**按量计费**。
2. 镜像选「基础镜像」→ PyTorch → 选 **2.1 以上 + Python 3.10 + CUDA 12.1** 的那个（名字里带 `pytorch2.x/cuda12.1` 即可）。
3. 创建实例 → 等状态变「运行中」→ 点「JupyterLab」进入，打开 Terminal。
4. 在 Terminal 里确认 GPU 可用：

```
nvidia-smi
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

预期输出第二行类似 `2.1.0+cu121 True`。**True 是硬性要求。**

注：「跑完就关机」规则从今天开始生效：每次离开终端前，在 AutoDL 控制台点**关机**（不是销毁！）。关机后只收少量磁盘费，重新开机数据还在。

任务 3：环境冒烟 wandb 上报（约 40 分钟）

1. 在 JupyterLab 里新建文件 `/root/env_check.py`，内容原样复制：

```
# env_check.py - 环境冒烟:GPU、矩阵乘、wandb 上报三件事
import torch, time, wandb

assert torch.cuda.is_available(), "CUDA 不可用,检查镜像是否选对"
print("torch:", torch.__version__, "| device:", torch.cuda.get_device_name(0))

run = wandb.init(project="week1-env", name="env-check", mode="online")
x = torch.randn(4096, 4096, device="cuda")
for i in range(10):
    t0 = time.time()
    y = x @ x
    torch.cuda.synchronize()
    wandb.log({"matmul_sec": time.time() - t0, "step": i})
    print(f"step {i}: {time.time()-t0:.3f}s")
run.finish()
print("ALL OK")
```

2. 终端里登录 wandb (粘贴你任务 1 存的 API Key) :

```
pip install wandb -q
wandb login <你的API Key>
```

3. 运行:

```
python env_check.py
```

完成判据: 终端打印 `ALL OK` ; 打开 `https://wandb.ai` 首页 → week1-env 项目 → 能看到 10 个 matmul_sec 数据点。

注: wandb login 报 network error 的话, 执行 `export WANDB_MODE=online` 再试; 仍不行就 `wandb login --host api.wandb.ai` 重来。超过 30 分钟没搞定, 记录问题到 NOTES.md, 继续任务 4 (wandb 可以明晚再补), 但**明天必须补上**。

任务 4: micrograd 视频上半场 (约 3.5 小时)

1. 打开视频: Karpathy 的 **「The spelled-out intro to neural networks and backpropagation: building micrograd」** (B 站搜「micrograd karpathy」有官方搬运, 或 YouTube 原链 `https://www.youtube.com/watch?v=VMj-3S1tku0`) 。
2. 今天只看 **00:00 到 1:05:00** (讲完 Value 类的 `+ * tanh` 与手工反向为止) 。剩下的明天的任务 1。
3. 看法要求 (**这是本任务的硬性要求**) : - 全程用电脑跟着, **他写一行你写一行**, 文件放 `E:\study\posttrain\day1\follow.py` 。 - 每个概念出现时暂停 30 秒, 用自己的话在 `E:\study\posttrain\NOTES.md` 里写一行 (中英文都行) 。 - 视频里用 `graphviz` 画计算图那段 (约 42 分钟处) : 跟着装 `pip install graphviz` 并把他的 `draw_dot` 函数抄进 `follow.py`; 本机渲染报错 (缺 Graphviz 系统包) 就**跳过画图**, 不要装系统包浪费时间。
4. 收工前在本地跑一遍 `follow.py`, 确认没有报错。

完成判据: `follow.py` 里有一个能前向计算的 `Value` 类雏形; `NOTES.md` 里有 ≥ 8 条自己的话。

注：今晚不需要看懂 100%。「_backward 为什么要这么写」这个问题如果卡住，写进 NOTES.md 带着睡，明天视频下半场会揭晓。

任务 5：收工笔记（约 10 分钟）

在 `E:\study\posttrain\NOTES.md` 里写三行：今天最容易的一个卡点、一个明天要回答的问题、明晚计划开始时间。

今日验收： wandb 有记录 + follow.py 存在 + NOTES.md ≥ 8 条。三项都完成才准睡。

Day 2 • 周二：micrograd 视频（下）+ 完整跟写

今日总览（6.5h）： 看完视频剩下的部分（拓扑排序、整个 MLP、训练循环），然后把 micrograd 的完整实现 **跟着视频逐行敲一遍**，晚上让它在二维数据集上训练收敛。今天结束时你会拥有一个「自己敲出来的、能训练的神经网络」——虽然它只有 40 行。

注：昨天的 GPU 实例今天用不上了（micrograd 是纯 CPU 项目），**记得把实例关机**。今天全部在本地写。

任务 1：视频下半场（约 2h）

- 从 **1:05:00** 继续看到结尾（约 2h25m 处结束）。今天覆盖：`Value.backward()` 的拓扑排序实现、`Neuron/Layer/MLP` 三个类、moons 数据集、训练循环。
- 继续「他写一行你写一行」，追加到 `day1/follow.py`。
- 视频结束前他把 loss 从 0.9 压到了 0.05 附近——你跟到同样的效果再停。

完成判据： follow.py 运行后 loss 逐步下降到 0.1 以下；NOTES.md 新增 ≥ 5 条（重点记「拓扑排序为什么必要」）。

任务 2：脱离视频整理成自己的三个文件（约 2.5h）

合掉视频，只对照 follow.py 和 micrograd 原仓库（<https://github.com/karpathy/micrograd>），把代码整理成自己的干净版本，放 `day2/` 下三个文件。

文件 1： `day2/engine.py`（Value 引擎，核心约 60 行）——先自己默写框架，卡住才看参考：

```

import math

class Value:
    """标量自动微分引擎:存 value、父节点、局部梯度函数"""
    def __init__(self, data, _children=(), _op=""):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += out.grad          # 加法:梯度原路分发
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")
        def _backward():
            self.grad += other.data * out.grad  # 乘法:交叉相乘
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def tanh(self):
        x = self.data
        t = (math.exp(2*x) - 1) / (math.exp(2*x) + 1)
        out = Value(t, (self,), "tanh")
        def _backward():
            self.grad += (1 - t**2) * out.grad  # tanh 导数 1-t^2
        out._backward = _backward
        return out

    def backward(self):
        # 拓扑排序:保证先算子节点再算父节点的梯度
        topo, visited = [], set()
        def build(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build(child)
            topo.append(v)
        build(self)
        self.grad = 1.0
        for v in reversed(topo):
            v._backward()

    def __repr__(self):
        return f"Value({self.data})"

```

注：__add__ 里 self.grad += 的 += 而不是 =，是因为一个节点可能被多个表达式共用（视频里的 bug 案例）。默写时这里错最多，注意。

文件 2： day2/nn.py （Neuron/Layer/MLP 三个类，约 40 行，照视频结构默写）。

文件 3： day2/train_moons.py ——用 sklearn 造 moons 数据训练：

```
import random
random.seed(42)
from sklearn.datasets import make_moons
from engine import Value
from nn import MLP

X, y = make_moons(n_samples=100, noise=0.15)
y = [2*yy - 1 for yy in y]          # 标签转 -1/+1
model = MLP(2, [16, 16, 1])         # 2入 16 16 1出

def loss_fn():
    losses = [(1 + -model([x1, x2])[0]).tanh() ** 2 for x1, x2 in X] # hinge 风格
    data_loss = sum(losses) / len(losses)
    reg = 1e-4 * sum(p * p for p in model.parameters())
    return data_loss + reg

for epoch in range(300):
    loss = loss_fn()
    for p in model.parameters():
        p.grad = 0.0
    loss.backward()
    lr = 1.0 - 0.9 * epoch / 300      # 学习率衰减
    for p in model.parameters():
        p.data -= lr * p.grad
    if epoch % 50 == 0:
        acc = sum((model([x1, x2])[0].data > 0) == yy for (x1, x2), yy in zip(X, y)) / len(y)
        print(f"epoch {epoch} loss {loss.data:.4f} acc {acc:.2%}")
```

完成判据： python train_moons.py 打印的最终 **acc ≥ 95%**（通常 300 轮内到 97%+）。

注：acc 卡在 50% 不动 20 分钟的话，99% 是梯度没清零或 backward 顺序错了——回看 engine.py 的 backward()。限时 40 分钟，解决不了就把两个文件与 follow.py 逐行 diff。

任务 3：加可视化（约 1h）

新建 day2/plot.py：训练完后用 matplotlib 把 100 个点和模型决策边界画出来（把平面 400 个网格点过一遍模型，plt.contourf）。存图 day2/moons.png。

完成判据： moons.png 能明显看到两条月牙被分界线分开。

任务 4：收工笔记（10 分钟）

NOTES.md 回答两个问题：①「梯度为什么要 += 而不是 =」②「backward 里为什么先 build 拓扑再 reversed」。

今日验收: moons.png + acc≥95% + NOTES ≥5 条新条目。

Day 3 • 周三：盲写复刻 + 数值梯度对拍

今日总览 (6h)： 全周最重要的三天之一。上午**关掉所有参考**，从空文件开始默写整个 micrograd (约 3h，卡住就记 NOTES 硬扛)；下午与原版 diff 修错、推 GitHub，再做数值梯度对拍单测。今天结束时你拥有一个**可以写进简历的 GitHub 仓库**。

注：盲写的目的不是背代码，是逼你暴露「以为懂了但没懂」的地方。**今天写得痛苦是正常的、是设计如此。**

任务 1：盲写 mine.py (约 3h, 含卡点记录)

1. 合掉所有窗口，只开一个空文件 `day3/mine.py` 和 `NOTES.md`。
2. 从零默写：Value 类 (add/mul/tanh/pow 可省/BACKWARD/拓扑 backward) + Neuron/Layer/MLP + 一个 30 轮训练循环 (数据用 `random` 造两个高斯团即可，不用 sklearn)。
3. 规则：
 - **不许打开任何参考文件/网页** (micrograd 仓库、`follow.py`、`engine.py` 全部关掉)。
 - 卡住超过 5 分钟 → 把「卡在哪、为什么」写进 `NOTES.md` 的 `## 盲写卡点` 小节，然后继续硬想；连续卡 15 分钟才允许瞄一眼自己的 `day2` (不是原仓库)。
4. 训练循环跑通、loss 下降即为盲写完成 (不要求 acc)。

完成判据： `mine.py` 独立运行且 loss 下降；`NOTES.md` 盲写卡点 ≥3 条 (**这些就是博客素材**)。

任务 2：diff 与修复 (约 1h)

1. 打开原仓库 <https://github.com/karpathy/micrograd/blob/master/micrograd/engine.py> 与你的 `mine.py` 对比。
2. 在 `NOTES.md` 写「三个差异」：命名差异、实现差异、我多写/少写了什么。
3. 修复所有语义 bug，重跑确认 loss 收敛。

任务 3：推 GitHub (约 40 分钟)

1. 新建仓库 `micrograd-from-scratch` (Public)，`day3/` 内容为根目录：`mine.py`、`train.py`、`gradcheck.py` (任务 4)、`README.md`。
2. `README` 写四段：这是什么 / 与原版的差异 / 怎么跑 / 学到什么 (从 `NOTES` 盲写卡点里挑 3 条)。
3. `git init && git add -A && git commit -m "micrograd from scratch" && git push` (仓库建好后页面上有现成命令)。

完成判据： GitHub 仓库链接能打开，`README` 渲染正常。**把链接记到 `NOTES.md` 顶部。**

任务 4：数值梯度对拍 gradcheck.py (约 1h)

数值梯度是验证一切自动微分的金标准。新建 `day3/gradcheck.py`：

```
"""数值梯度 vs 反向传播 对拍:两者误差应 < 1e-5"""
import random, math
from mine import Value

random.seed(0)

def numerical_grad(f, x, h=1e-6):
    return (f(x + h).data - f(x - h).data) / (2 * h)

def f(x):
    return (x * 3 + Value(2)) * x + x.tanh() # 随便一个复合函数

x = Value(0.7)
f(x).backward()
analytic = x.grad
numeric = numerical_grad(lambda t: f(t), x)
diff = abs(analytic - numeric)
print(f"analytic={analytic:.8f} numeric={numeric:.8f} diff={diff:.2e}")
assert diff < 1e-5, f"梯度对拍失败 diff={diff}"
print("GRADCHECK OK")
```

完成判据： `python gradcheck.py` 打印 `GRADCHECK OK`。把这个文件也 push 上 GitHub。

任务 5：收工笔记 (10 分钟)

NOTES.md 写：盲写时最痛的 1 个点 + 为什么数值梯度能当裁判（一句话）。

今日验收： mine.py 收敛 + GitHub 链接 + gradcheck OK + 盲写卡点 ≥ 3 条。

Day 4 • 周四：限时自测 + 错题映射 + 博客成文

今日总览 (6h)： 今天不动代码主线。上午闭卷自测 6 道基础题 (90 分钟)，暴露「视频看懂了但手推写不出」的空洞；下午把错题映射到具体补课日期，然后把前三天的素材（盲写卡点、对拍、决策边界图）写成知乎文章草稿。**输出物是后训练求职的门面，不能省。**

任务 1：闭卷自测 90 分钟（写在本子上或 md 里，不查任何资料）

- 手推反向传播：网络 $y = \tanh(w_2 \cdot \tanh(w_1 \cdot x + b_1) + b_2)$ ，写出 $\text{loss}=L$ 对 w_1 的梯度表达式（链式法则逐步展开，标注每步维度）。
- 写出 softmax 的定义；证明 softmax + 交叉熵的梯度是 $p - y$ ；说明为什么实现时要把它们合并成一个算子。
- 写出 KL 散度定义 $D_{\text{KL}}(P||Q)$ ；说明它与交叉熵、熵的关系；为什么训练时用 CE 而不是直接最小化 KL。

- 画出 Transformer decoder block 的结构图（至少标出：LayerNorm、多头注意力、FFN、残差连接的位置），并写出每个子层的输入输出维度关系。
- 写出 Adam 的两个动量更新式与最终参数更新式；解释为什么要做偏差修正。
- 用伪代码（NumPy 风格）写单头 self-attention 的前向（Q K V softmax 加权），并指出 causal mask 应该加在哪一行。

完成判据：6 题全部写了答案（写错没关系，**空着才算失败**）。计时 90 分钟，到点停笔。

任务 2：对答案 + 错题映射（约 1.5h）

- 参考答案要点：- 第 1 题： $dL/dw_1 = dL/dy \cdot (1-y^2) \cdot w_2 \cdot (1-h^2) \cdot x$ ，其中 $h=\tanh(w_1x+b_1)$ 、 $y=\tanh(\cdot)$ 。维度全部与各自变量同形。- 第 2 题： $\text{softmax}_i = e^{z_i} / \sum e^{z_j}$ ；合并算子是因为分开实现数值不稳定（上溢）且梯度可简化为 $p-y$ 。- 第 3 题： $D_{KL}(P||Q) = \sum P \log(P/Q) = H(P,Q) - H(P)$ ；P 固定时最小化 KL $\equiv$ 最小化交叉熵。- 第 4 题： $x \rightarrow \text{LN} \rightarrow \text{QKV 线性} \rightarrow \text{多头注意力} \rightarrow \text{残差相加} \rightarrow \text{LN} \rightarrow \text{FFN} \rightarrow \text{残差相加}$ ；全程 (B,T,C) 形状不变（ $C \rightarrow 4C \rightarrow C$ 的瓶颈在 FFN 内部）。- 第 5 题： $m = \beta_1 \cdot m + (1-\beta_1)g$ ； $v = \beta_2 \cdot v + (1-\beta_2)g^2$ ； $\hat{m} = m / (1-\beta_1^t)$ ； $\hat{v} = v / (1-\beta_2^t)$ ； $\theta = \text{lr} \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ 。偏差修正是因为零初始化导致前期估计偏小。- 第 6 题： $\text{scores} = QK^T / \sqrt{d}$ ；mask 行加 $-\text{inf}$ （softmax 前）； $\text{attn} = \text{softmax}(\text{scores}) @ V$ 。mask 加在 softmax **之前**。
- 新建 `day4/week1-gaps.md`，每条错题一行，格式：**「错了什么 → 反映哪个知识空洞 → 补在何时」**。例如：
「Adam 偏修正写不出 → 优化器动量统计没吃透 → Day 6 任务 1 补 + Week2 复习」。- **硬性要求：至少 6 条，每条都必须映射到一个具体日期**（Day 5/6 内的某任务，或「Week2-Dx」）。

任务 3：知乎第 0 篇成文（约 2h）

- 打开 <https://zhuanlan.zhihu.com> → 写文章。
- 标题（三选一或自拟）：「我花三天盲写了一个 micrograd」「别再看视频了：手写反向传播的三个卡点」「从零手写 autograd：梯度为什么会 +=」。
- 结构照抄（总 1500-2500 字）：① 为什么要手写（2 段）② 盲写卡点 top3（用 NOTES.md 素材，贴关键代码 ≤ 3 段）③ 数值梯度对拍（贴 gradcheck 输出）④ 决策边界图（贴 moons.png）⑤ 下一步（attention）。
- 先存草稿，明天任务 5 发布。

完成判据：草稿保存成功，图片 ≥ 2 张（moons.png + 截图任意）。

任务 4：收工笔记（10 分钟）

NOTES.md：今天自测暴露的最大空洞一句话 + 「它会被哪天补上」。

今日验收：自测 6 题写完 + gaps ≥ 6 条且每条有日期 + 博客草稿保存。

Day 5 • 周五：手写 attention + 博客发布

今日总览 (6h)：今天从 micrograd 跨到 Transformer 的心脏——注意力。纯 NumPy/纯 Python 手写，不依赖 torch。四个任务：单头 → 多头 → 形状与因果性单测 → 热力图可视化。晚上 30 分钟把博客发出去。

任务 1：原理预热 (45 分钟)

1. 读一遍下面的极简解释（这就是全部数学，没有更多）：

给定输入矩阵 X (T 个 token，每个 C 维)，三个可学习矩阵 W_Q 、 W_K 、 W_V 生成查询/键/值。每个 token 拿自己的 Q 去和所有 token 的 K 做点积（相似度），除以 $\sqrt{d}$ 缩放，加 causal mask（第 i 行第 $j \geq i$ 列置 $-\infty$ ，防止偷看未来），softmax 成权重，再对 V 加权求和——输出就是「每个位置对它之前所有信息的加权摘要」。

2. 在 NOTES.md 默写这条流水线 ($QK^T \rightarrow \text{scale} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow @V$)，写不出看一眼再写，直到能独立写出。

任务 2：单头 + 多头实现 (约 2.5h)

新建 `day5/attention.py`，按 TODO 填空（**TODO 是你的作业，先自己写再对答案**）：

```
import math
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)    # 数值稳定
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def single_head_attention(X, Wq, Wk, Wv, causal=True):
    """X:(T,C)  Wq/Wk/Wv:(C,d)  返回 (T,d)"""
    Q = X @ Wq                                # TODO 1: 生成 Q
    K = X @ Wk                                # TODO 2
    V = X @ Wv                                # TODO 3
    scores = Q @ K.T / math.sqrt(Q.shape[-1]) # TODO 4: 缩放点积
    if causal:                                 # TODO 5: causal mask
        T = X.shape[0]
        scores += np.triu(np.full((T, T), -np.inf), k=1)
    A = softmax(scores)                        # TODO 6
    return A @ V, A                           # A 留着可视化

def multi_head_attention(X, Ws, Wo, causal=True):
    """Ws: 列表,每个元素是 (Wq,Wk,Wv);Wo:(h*d, C)。各头结果拼接后过 Wo"""
    heads, amaps = [], []
    for Wq, Wk, Wv in Ws:
        out, A = single_head_attention(X, Wq, Wk, Wv, causal)
        heads.append(out); amaps.append(A)
    return np.hstack(heads) @ Wo, amaps
```

完成判据：不看答案独立填完 6 个 TODO（答案就是上面注释里的行，先写再对）。

任务 3: 单测 (约 1.5h)

新建 `day5/test_attention.py` , 写 4 个断言测试并全部通过:

```
from attention import single_head_attention, multi_head_attention
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(5, 8))          # T=5, C=8

def test_shape():
    out, A = single_head_attention(X, rng.normal(size=(8, 6)), rng.normal(size=(8, 6)), rng.normal(size=(8, 6)))
    assert out.shape == (5, 6) and A.shape == (5, 5)

def test_causal():
    _, A = single_head_attention(X, rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=(8,4)))
    assert np.allclose(np.triu(A, k=1), 0)      # 上三角(未来位)权重必须为 0

def test_rows_sum_one():
    _, A = single_head_attention(X, rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=(8,4)))
    assert np.allclose(A.sum(axis=-1), 1)      # 每行权重和为 1

def test_multi_head():
    Ws = [(rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=(8,4))) for _ in range(3)]
    out, _ = multi_head_attention(X, Ws, rng.normal(size=(12, 8)))
    assert out.shape == (5, 8)

for t in [test_shape, test_causal, test_rows_sum_one, test_multi_head]:
    t(); print(t.__name__, "PASS")
print("ALL TESTS PASS")
```

完成判据: 打印 4 个 PASS + `ALL TESTS PASS`。

任务 4: 热力图可视化 (约 1h)

新建 `day5/visualize.py` : 用 matplotlib 把一个头的注意力矩阵 A 画成 5×5 热力图 (`plt.imshow(A)`), 保存 `day5/attn.png` 。看图确认两件事写进 NOTES.md: ① 上三角是黑的 (全 0) ② 每行最亮的格子通常在对角线附近。

任务 5: 博客发布 (30 分钟)

打开昨天草稿, 补一段「下一步:手写 attention」+ `attn.png` , 点发布。把链接贴到 **NOTES.md 顶部**。

今日验收: 4 个单测 PASS + `attn.png` + 博客已发布 (有链接) 。

Day 6 • 周六: PyTorch 桥接 + 对拍 + 周复盘

今日总览 (7h)：今天把手写的世界和 PyTorch 的世界接起来——用 `torch.nn` 重写 moons 分类器和 attention，并与自己的手写版对拍（数值一致是硬判据）。晚上做周复盘。这是「我懂原理」到「我会用框架」的最后一公里。

注：今天需要 GPU 吗？不需要，torch CPU 版足够（`pip install torch --index-url https://download.pytorch.org/whl/cpu`）。

任务 1：概念对照表 (45 分钟)

在 NOTES.md 写一张表（先默写后对照，写不出=自测第 5 题的空洞没补上）：

micrograd	PyTorch	一句话关系
Value.data	tensor (requires_grad)	都存数值
Value.grad	tensor.grad	反向后自动填充
backward() 拓扑循环	loss.backward()	torch 内置了拓扑排序
MLP 类	nn.Module + nn.Linear	torch 帮你管参数注册
p.data -= lr*p.grad	torch.optim.SGD.step()	优化器抽象

任务 2：torch 重写 moons 分类器 (约 2h)

新建 `day6/train_moons_torch.py`：用 `nn.Linear` + `tanh` 搭 2-16-16-1 网络，`BCEWithLogitsLoss` 或 `hinge`、`torch.optim.SGD` 或 `Adam`，训练到 `acc ≥ 95%`。要求：

- 与 day2 相同的数据（`make_moons(n_samples=100, noise=0.15, random_state=42)`）。
- 代码量预期 ≤50 行（体会框架帮你省了什么）。

完成判据：acc ≥ 95%；NOTES.md 写两行「torch 帮我做了什么/我要自己做什么」。

任务 3：torch attention 与手写版对拍 (约 1.5h)

新建 `day6/attention_torch.py`：

```

import torch, torch.nn.functional as F, math
import numpy as np
from day5_attention import single_head_attention # 把 day5 目录加入 sys.path 或复制文件

def torch_attention(X, Wq, Wk, Wv):
    X, Wq, Wk, Wv = map(lambda t: torch.tensor(t, dtype=torch.float64), (X, Wq, Wk, Wv))
    T = X.shape[0]
    A = torch.softmax(X @ Wq @ (X @ Wk).T / math.sqrt(Wq.shape[1])
                      + torch.triu(torch.full((T, T), float("-inf")), 1), dim=-1)
    return (A @ (X @ Wv)).numpy(), A.numpy()

# 同一组权重喂给两个实现,输出应 allclose
rng = np.random.default_rng(1)
X = rng.normal(size=(6, 8))
Wq, Wk, Wv = (rng.normal(size=(8, 5)) * 0.5 for _ in range(3))
mine, _ = single_head_attention(X, Wq, Wk, Wv, causal=True)
ref, _ = torch_attention(X, Wq, Wk, Wv)
assert np.allclose(mine, ref, atol=1e-10), f"对拍失败 max_diff={np.abs(mine-ref).max()}"
print("ATTENTION CROSSCHECK OK")

```

完成判据：打印 `ATTENTION CROSSCHECK OK`（浮点误差容忍 `atol=1e-10` 内）。push 到 GitHub 仓库。

任务 4：周复盘 review.md (约 1.5h)

新建 `day6/review.md`，写五块：① 本周硬产出清单（贴链接/截图路径）② 自测 6 题现在还有哪题不会 ③ 一周时间真实花销（对照 41.5h）④ 最有价值的一件事 ⑤ 下周最想补的一个洞。

任务 5：gaps 清账 (约 1h)

过一遍 `day4/week1-gaps.md`，每条已补的标 `[x]` 并写一句「怎么补的」；没补的确认都映射到了下周。

任务 6：缓冲 (约 0.5h，多了就提前收工)

今日验收：torch moons `acc` $\geq 95\%$ + CROSSCHECK OK + review.md + gaps 全部有状态。

Day 7 • 周日：缓冲补漏 + 总验收 + 下周预习

今日总览 (4h)：今天不排新内容。上午按 gaps 清单补漏，中午做总验收，下午 30 分钟预习下周。之后——彻底休息。

任务 1：补漏 (约 2h)

打开 `week1-gaps.md` 和 `review.md`，按优先级清洞。**规则：最多补 2 个洞，贪多必失。**

任务 2：总验收（30 分钟）

逐项勾选（输出到 review.md 末尾）：

#	验收项	判据
1	GPU 环境	wandb week1-env 有记录；知道怎么开/关机
2	micrograd 跟写	day2 三文件齐全，moons acc≥95%
3	盲写复刻	GitHub 仓库可访问，mine.py 收敛，gradcheck OK
4	知识自测	week1-gaps.md ≥6 条且全部映射日期
5	公开输出	知乎文章链接可访问
6	手写 attention	4 单测 PASS + CROSSCHECK OK
7	周复盘	review.md 五块齐全

≥6 项通过 = 本周合格；不足 6 项，把缺的写进下周 Day 1 的前 2 小时。

任务 3：下周预习（30 分钟）

打开 Karpathy 的下一个视频「The spelled-out intro to language modeling (makemore part 1)」看前 20 分钟，只做一件事：搞清楚 makemore 和 micrograd 的关系（bigram 语言模型是下一周的起点）。

今日验收：review.md 里验收表 ≥6 项打勾。

三、常见问题与纪律

- 某天没完成怎么办？不要熬夜追。把未完成项按「天-任务号」写进 gaps.md，Day 6 的缓冲与 Day 7 就是给它们准备的。连续两天全崩则砍掉 Day 5 的可视化任务（保主线）。
- 卡住多久该求助？单点超过 40 分钟：先写 NOTES 记录，然后搜（Google/官方文档/原仓库 issue），再 20 分钟仍无解就跳过并在 gaps 里立条目。跳过不丢人，欠账不留名才可怕。
- GPU 费用：只在 Day 1-2 用卡，每天结束关机（不是销毁）；本周总花费应 ≤¥15。
- 时间不够 6 小时的天：优先保「硬产出」栏里的东西；视频中可 1.25 倍速，但代码跟写段落必须原速。